

Chapter Overview

Chapter 10 covers search and Elasticsearch in ShopVerse: index mapping, analyzers, query DSL, full-text search, faceted search, autocomplete, aggregations, relevance scoring (BM25), Spring Data Elasticsearch integration, data synchronisation via Debezium CDC, nested queries, percolator, ILM, performance tuning, monitoring, and comparison with alternative search engines.

Sub-Chapter	Topic
10-01	Elasticsearch Fundamentals – index, document, shard, replica, cluster
10-02	Index Mapping & Analyzers – text vs keyword, english analyzer, edge_ngram
10-03	Query DSL – bool, match, term, range, filter vs query context
10-04	Full-Text Search – multi_match, boosting, fuzziness, highlight
10-05	Faceted Search & Aggregations – terms, range, nested aggs
10-06	Autocomplete & Suggest – completion, search_as_you_type, edge_ngram
10-07	Relevance Scoring – BM25, function_score, field boost
10-08	Spring Data Elasticsearch – @Document, ElasticsearchRepository, NativeQuery
10-09	CDC with Debezium – Postgres WAL, Kafka Connect, index sync
10-10	Advanced Queries – nested objects, percolator, more-like-this
10-11	Index Templates & ILM – rollover, hot/warm/cold/delete phases
10-12	Performance & Monitoring – bulk API, refresh_interval, cluster health

Concept	ES Term	SQL Equivalent	Notes
Collection of docs	Index	Table	products, orders, reviews indexes
JSON object	Document	Row	One product = one doc
JSON field	Field	Column	name, price, category, tags
Field type defs	Mapping	Schema	Defined upfront or auto-detected
Subset of index data	Shard	Partition	Split for parallelism; default 1 primary shard
Copy of shard	Replica	Read replica	Fault tolerance + read scaling
ES process	Node	DB server	Data, master, coordinator node roles
Group of nodes	Cluster	DB cluster	ShopVerse: 3 data + 1 master node

Cluster Architecture

Elasticsearch Cluster – shopverse-es

Dedicated Master Node – coordinates cluster state; no data storage

Data Node 1 – P0 primary + P1 replica shards

Data Node 2 – P1 primary + P0 replica shards

Data Node 3 – hot-warm tier; archived indexes

Ingest Node – pre-processes documents via ingest pipelines

Index: products – 2 primary shards × 1 replica = 4 shard copies total

Node Role	Responsibilities	Heap Recommendation
Master-eligible	Cluster state, index creation, shard allocation	4 GB – low heap needed
Data (hot)	Stores & serves active shards; I/O intensive	16–32 GB; fast NVMe SSD
Data (warm)	Read-only older shards; less I/O	8–16 GB; HDD acceptable
Coordinating	Routes queries to shards, merges results; 80% CPU	8 GB; high network bandwidth

Products Index Mapping (products_v1.json)

```
{ "mappings": { "properties": {
  "id": { "type": "keyword" },
  "name": { "type": "text", "analyzer": "english",
  "fields": { "keyword": { "type": "keyword" } },
  "suggest": { "type": "search_as_you_type" } } },
  "description": { "type": "text", "analyzer": "english" },
  "category": { "type": "keyword" },
  "brand": { "type": "keyword" },
  "price": { "type": "float" },
  "tags": { "type": "keyword" },
  "isActive": { "type": "boolean" },
  "ratings_avg": { "type": "float" },
  "createdAt": { "type": "date" },
  "suggest": { "type": "completion" }
} } }
```

text vs keyword

Field Type	Analyzed?	Aggregation?	Search Type	Example Fields
text	Yes (tokenised, lowercased, stemmed, etc.)	No (use keyword aggregation)	Fuzzy / prefix match	name, description
keyword	No (exact string)	Yes	Exact / terms agg	category, brand, status

Analyzer Pipeline

Analyzer = Character Filters → Tokenizer → Token Filters

Built-in: english analyzer

Tokenizer: standard (whitespace + punctuation split)

Token Filters: lowercase → stop_words → porter_stem

Input "Running Shoes for Athletes" → [run, shoe, athlete]

Custom: shopverse_autocomplete (edge_ngram)

min_gram=1, max_gram=15, side=front

Input "Blue" → [b, bl, blu, blue]

Filter vs Query Context

Context	Affects Score	Cached	Best For
Query (must/should/must_not)	Yes – contributes to score	No	Full-text relevance: name, desc
Filter (filter/must_not)	No – binary match	Yes	Exact: isActive, category, price range

```
// Canonical ShopVerse product search query
{ "query": { "bool": {
  "must": [
    { "multi_match": { "query": "wireless headphones",
      "fields": ["name^3", "description", "tags^2", "brand^2"],
      "fuzziness": "AUTO", "minimum_should_match": "75%" } }
  ],
  "filter": [
    { "term": { "isActive": true } },
    { "term": { "category": "electronics" } },
    { "range": { "price": { "gte": 1000, "lte": 50000 } } }
  ]
} } }
```

Common Query Types Reference

Query	Purpose	ShopVerse Example
match	Full-text with analysis	name: "wireless headphones"
multi_match	Full-text across multiple fields	name^3, description, tags^2
term	Exact match (no analysis)	category: "electronics"
terms	Multiple exact values (OR)	tags: ["wireless", "premium", "featured"]
range	Numeric / date range	price: >=1000 AND <=5000
bool	Compose queries	must+filter+should+must_not clauses
match_phrase	Phrase in order	description: "noise cancelling"
fuzzy	Typo tolerance (edit distance)	name: "heaphones" → headphones
exists	Non-null field	filter: field "discount" exists
ids	Fetch by document IDs	fetch product IDs from Redis then enrich via ES

NativeQuery in Spring Data Elasticsearch

```
@Service public class ProductSearchService {
    private final ElasticsearchTemplate template;

    public SearchPage search(SearchCriteria c, Pageable p) {
        Query q = NativeQuery.builder()
            .withQuery(qb -> qb.bool(b -> b
                .must(m -> m.multiMatch(mm -> mm
                    .query(c.getKeyword())
                    .fields("name^3", "description", "tags^2", "brand^2")
                    .fuzziness(Fuzziness.AUTO)
                    .minimumShouldMatch("75%"))))
                .filter(f -> f.term(t -> t.field("isActive").value(true)))
                .filter(f -> f.range(r -> r.field("price")
                    .gte(JsonData.of(c.getMinPrice()))
                    .lte(JsonData.of(c.getMaxPrice())))))
            .withHighlightQuery(new HighlightQuery(
                new Highlight(List.of(
                    new HighlightField("name"),
                    new HighlightField("description")), null))
            .withPageable(p)
            .build();
        SearchHits hits = template.search(q, ProductDocument.class);
        return SearchHitSupport.searchPageFor(hits, p);
    }
}
```

Highlighting & Snippet Response

Parameter	Value	Effect
pre_tags		Opening HTML tag for matched term
post_tags		Closing HTML tag for matched term
fragment_size	150 chars	Snippet length returned per field
number_of_fragments	3	Max snippets per field in response

```
// Single request: search results + facet counts
NativeQuery facetQ = NativeQuery.builder()
.withQuery(q -> q.bool(b -> b
.filter(f -> f.term(t -> t.field("isActive").value(true))))))
.withAggregation("categories",
Aggregation.of(a -> a.terms(t -> t.field("category").size(20))))
.withAggregation("brands",
Aggregation.of(a -> a.terms(t -> t.field("brand").size(20))))
.withAggregation("price_ranges",
Aggregation.of(a -> a.range(r -> r.field("price")
.ranges(v -> v.to(JsonData.of(1000)),
v -> v.from(JsonData.of(1000)).to(JsonData.of(5000)),
v -> v.from(JsonData.of(5000))))))
.withAggregation("avg_rating",
Aggregation.of(a -> a.avg(av -> av.field("ratings_avg"))))
.withPageable(PageRequest.of(0, 20)).build();
```

Aggregation Types

Type	Example	Returns
terms	field: category, size: 20	Top 20 categories with doc count each
range	field: price, [0-1k, 1k-5k, 5k+]	Doc count per price bucket
date_histogram	field: createdAt, cal_interval: month	Products per calendar month
avg / sum / min / max	field: ratings_avg	Single numeric metric across result set
nested	path: variants + sub-aggregation	Agg on nested array objects (variants)
global	{}	Break out of filter scope; agg all docs regardless of query

Completion Suggester (FST – fastest)

```
"suggest": { "type": "completion", "analyzer": "standard" }
// Index-time: populate suggest inputs when saving product
"suggest": {
  "input": ["Sony WH-1000XM5", "Sony Headphones", "Wireless ANC Headphones"],
  "weight": salesCount // higher = ranked first in suggestions
}
// Query-time: prefix "son"
{ "suggest": { "product-suggest": {
  "prefix": "son",
  "completion": { "field": "suggest", "size": 5,
  "fuzzy": { "fuzziness": 1 } }
} } }
```

search_as_you_type Field

```
"name": { "type": "text", "fields": {
  "suggest": { "type": "search_as_you_type" }
} }
// Auto-creates shingle sub-fields: suggest._2gram, ._3gram, ._index_prefix
{ "multi_match": { "query": "son wir",
  "type": "bool_prefix",
  "fields": ["name.suggest", "name.suggest._2gram", "name.suggest._3gram"] } }
```

Autocomplete Strategy Comparison

Strategy	Index Size	Query Speed	Typo Tolerance	Best For
Completion suggester	Small (FST)	O(1) – fastest	Yes (fuzzy option)	Pure autocomplete; catalog > 100k
search_as_you_type	Medium	Fast (index scan)	Limited	Instant search with BM25 ranking
edge_ngram (custom)	Large (n tokens per word)	Fast	No	Custom analyzer pipeline needed

BM25 Score Components

Factor	Formula Concept	Effect on Score
TF (Term Frequency)	Saturating fn of count in doc	More occurrences = higher; diminishing returns
IDF (Inverse Doc Freq)	$\log(1 + (N-n+0.5)/(n+0.5))$	Rare term across corpus = higher IDF
Field length norm	$1 / \sqrt{\text{numTokens}}$	Match in short field (title) > long field (description)
Field boost	× multiplier on field	$\text{name}^3 = 3 \times \text{weight}$ vs unweighted field

```
// function_score: BM25 relevance × business rules
{ "query": { "function_score": {
  "query": { "bool": { "must": [{ "multi_match": { "query": "headphones",
"fields": ["name^3", "description", "tags^2"] } } ] },
"filter": [{ "term": { "isActive": true } } ] } },
"functions": [
{ "field_value_factor": {
  "field": "salesCount", "factor": 0.1, "modifier": "log2p",
  "missing": 1 } },
{ "gauss": {
  "createdAt": { "origin": "now", "scale": "30d", "decay": 0.5 } } },
{ "filter": { "term": { "isFeatured": true } }, "weight": 2 }
],
"boost_mode": "multiply",
"score_mode": "sum"
} } }
```

boost_mode	Effect
multiply (default)	Final score = query_score × functions_sum
replace	Ignore query score; use only functions
sum	Add functions score to query score
avg	Average of query score and functions


```

@Document(indexName = "products", createIndex = false)
public class ProductDocument {
    @Id private String id;
    @Field(type=FieldType.Text, analyzer="english")
    private String name;
    @Field(type=FieldType.Text, analyzer="english")
    private String description;
    @Field(type=FieldType.Keyword) private String category;
    @Field(type=FieldType.Keyword) private String brand;
    @Field(type=FieldType.Float) private Float price;
    @Field(type=FieldType.Keyword) private List tags;
    @Field(type=FieldType.Boolean) private Boolean isActive;
    @Field(type=FieldType.Float) private Float ratingsAvg;
    @Field(type=FieldType.Date, format=DateFormat.date_time) private Instant createdAt;
    @CompletionField private Completion suggest;
}

public interface ProductSearchRepo
extends ElasticsearchRepository {
    Page findByCategoryAndIsActiveTrue(String cat, Pageable p);
    List findByPriceBetween(float min, float max);
}

```

Repository Method	Auto-generates	Limitation
findByCategory(...)	Term query on category field	Keyword field only – no full-text
findByPriceBetween(a,b)	Range query on price	Simple numeric range
NativeQuery via template	Any ES query DSL	Requires ElasticsearchTemplate injection

Change Data Capture Architecture

PostgreSQL – Write-Ahead Log (WAL) – source of truth

Debezium Postgres Connector (Kafka Connect) – reads WAL via replication slo

Publishes to Kafka topic: shopverse.public.products

Event payload: { op: c/u/d, before: {...}, after: {...}, ts_ms: ... }

Kafka Topic: shopverse.public.products (retention: 7 days)

ProductIndexConsumer – @KafkaListener – processes change events

op=c (create) or op=u (update) → upsert document in ES

op=d (delete) → deleteById from ES index

Elasticsearch products index – eventually consistent (<200ms lag)

```
// Debezium Kafka Connect config snippet
{ "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
  "database.hostname": "postgres-primary",
  "database.dbname": "shopverse",
  "table.include.list": "public.products,public.categories",
  "slot.name": "debezium_shopverse",
  "publication.name": "dbz_publication",
  "transforms": "unwrap",
  "transforms.unwrap.type": "io.debezium.transforms.ExtractNewRecordState" }
// Consumer
@KafkaListener(topics="shopverse.public.products", groupId="es-indexer")
public void onProductChange(ProductChangeEvent evt) {
    if ("d".equals(evt.getOp()))
        repo.deleteById(String.valueOf(evt.getBefore().getId()));
    else
        repo.save(mapper.toDocument(evt.getAfter()));
}
```

Nested Objects – Product Variants

```
"variants": { "type": "nested", "properties": {  
  "sku": { "type": "keyword" },  
  "color": { "type": "keyword" },  
  "size": { "type": "keyword" },  
  "stock": { "type": "integer" }  
} }  
// Find products with BLACK, IN-STOCK variant  
{ "query": { "nested": {  
  "path": "variants",  
  "query": { "bool": { "must": [  
    { "term": { "variants.color": "Black" } },  
    { "range": { "variants.stock": { "gt": 0 } } }  
  ] } },  
  "inner_hits": { "size": 3 }  
} } }
```

Percolator – Reverse Search / Saved Alerts

```
"registered_query": { "type": "percolator" }  
// Store query: user wants alert when new 4K TV < 30000  
PUT /alerts/_doc/user123-4k-tv  
{ "registered_query": {  
  "bool": {  
    "must": [{ "match": { "name": "4K TV" } }],  
    "filter": [{ "range": { "price": { "lte": 30000 } } }]  
  }  
}  
// Check new product against all stored user queries  
POST /alerts/_search  
{ "query": { "percolate": {  
  "field": "registered_query",  
  "document": { "name": "Samsung 55-inch 4K TV", "price": 28000 }  
} } }
```

Index Lifecycle Management Phases

Phase	Trigger	Actions	ShopVerse Use Case
Hot	Active writes + reads	Rollover: 50 GB or 30 days; 1 replica	Current month products, active orders
Warm	After rollover	Force merge → 1 segment; replica	Last 3 months order events
Cold	60 days old	Searchable snapshot (mount from S3)	Archived order history
Frozen	120 days old	Partially unmounted; slow; cheap	Audit logs seldom accessed
Delete	365 days old	delete_index	Log indexes older than 1 year

```
// ILM policy definition
PUT /_ilm/policy/shopverse-logs-policy
{ "policy": { "phases": {
  "hot": { "actions": { "rollover": { "max_size": "50gb", "max_age": "30d" } } },
  "warm": { "min_age": "30d", "actions": { "forcemerge": { "max_num_segments": 1 } } },
  "cold": { "min_age": "60d", "actions": { "searchable_snapshot": { "snapshot_repository":
    "s3-backup" } } },
  "delete": { "min_age": "365d", "actions": { "delete": {} } }
} } }
```

Index Template

```
PUT /_index_template/shopverse-products
{ "index_patterns": ["products-*"],
  "template": {
    "settings": { "number_of_shards": 2, "refresh_interval": "5s",
      "index.lifecycle.name": "shopverse-products-policy",
      "index.lifecycle.rollover_alias": "products" },
    "mappings": { "properties": { ... } }
  }
}
```

Setting	Default	Recommendation	Reason
<code>refresh_interval</code>	1s	5s prod; -1 bulk load	1s = new Lucene segment/s; -1 = no refresh during bulk
<code>number_of_shards</code>	1	2 for large index	Parallelism; cannot change after creation
<code>number_of_replicas</code>	1	0 bulk; 1 prod	Skip replica sync during bulk = 2-5x faster
<code>bulk batch size</code>	—	1 000 – 5 000 docs	Too large = heap pressure; too small = overhead
<code>heap size</code>	JVM default	50% of RAM, max 32 GB	32 GB disables compressed OOPs
<code>indices.memory.index_buffer_size</code>	10%	20% for write-heavy	Larger buffer = less frequent Lucene flushes

Zero-Downtime Reindex with Alias

```
# 1. Create new index with corrected mapping
PUT /products-v2 { "mappings": { ... } }
# 2. Reindex data asynchronously
POST /_reindex?wait_for_completion=false
{ "source": { "index": "products-v1" }, "dest": { "index": "products-v2" } }
# 3. Wait for task completion (poll /_tasks/)
# 4. Atomically switch alias
POST /_aliases { "actions": [
  { "remove": { "index": "products-v1", "alias": "products" } },
  { "add": { "index": "products-v2", "alias": "products" } }
] }
```

Key Monitoring APIs

API	Command	What to Watch
Cluster health	GET <code>/_cluster/health</code>	status: green/yellow/red; unassigned_shards
Node stats	GET <code>/_nodes/stats</code>	JVM heap_used_percent; search.query_time_in_millis
Index stats	GET <code>/products/_stats</code>	indexing.index_total; search.query_total
Slow log	<code>index.search.slowlog.threshold.query</code>	Log queries slower than threshold

■ Do

- ✓ Use filter context (filter/must_not) for exact matches – binary result, cached, much faster.
- ✓ Use query context (must/should) only for full-text relevance so caching isn't wasted.
- ✓ Boost title field (name^3) – a title match should outrank a body-text match for the same term.
- ✓ Set refresh_interval=-1 and replicas=0 during bulk load; restore to 5s / 1 after – 5× faster.
- ✓ Use index aliases for zero-downtime reindex: create new, reindex, atomically switch alias.
- ✓ Use Debezium CDC for Postgres→ES sync – eliminates dual-write inconsistency on failure.
- ✓ Use completion suggester for autocomplete – FST in memory, O(1) lookup, most efficient.
- ✓ Monitor cluster health every minute; alert on yellow (unassigned replicas) before red (data loss).
- ✓ Define index templates before creating indexes – ensures consistent settings/mappings.
- ✓ Limit shard count: aim for shard size 10–50 GB; over-sharding hurts coordination overhead.

■ Anti-Patterns

- Using Elasticsearch as primary data store – eventually consistent; Postgres is master.
- Forgetting isActive filter – search returns deleted/inactive products without it.
- Using text field for aggregations – use .keyword sub-field; text is analyzed, not aggregatable.
- Over-sharding – many tiny shards add inter-node coordination overhead; start small.
- Dual-write from app to Postgres AND ES – inconsistent on partial failure; use Debezium CDC.
- Ignoring refresh_interval during bulk load – default 1s forces a Lucene flush per second.
- Storing large binary data (images) in ES documents – index bloat; store URLs not blobs.
- Using wildcard queries on un-analyzed fields at scale – full scan of every term in index.

Sub-Chapter	Key Point	One-Liner
10-01 Fundamentals	Shards = Lucene index	1 replica = fault tolerance + read scale; 2 shards = parallelism
10-02 Mapping	text vs keyword	text: analyzed full-text. keyword: exact, aggregatable.
10-03 Query DSL	Disjunctive filter cached	filter for exact matches; must/should for scored full-text
10-04 Full-text	multi_match + name^3	Title match outranks description match by 3×
10-05 Facets	terms agg	Returns counts per category/brand alongside search results
10-06 Autocomplete	completion type (FST)	O(1) lookup; most efficient autocomplete strategy
10-07 BM25	IDF × TF × norm	Rare term in short field = highest BM25 score
10-08 Spring CD	CD ES, NativeQuery	ElasticsearchTemplate for custom DSL queries
10-09 CDC	Debezium WAL reader	WAL → Kafka → consumer → ES upsert; no dual-write
10-10 Advanced	nested type	Nested prevents cross-object field matching in arrays
10-11 ILM	hot/warm/cold/delete	Rollover at 50 GB; cold = searchable snapshot on S3
10-12 Perf	refresh_interval=-1 bulk	Reset to 5s after; bulk API for batch indexing

Interview Quick-Check

Question	Concise Answer
Query vs filter context?	Query: scored (_score affected), not cached. Filter: binary, cached → faster. Use filter for isActive, c
How does BM25 score?	TF (occurrence count, saturating) × IDF (rarity across index) × length norm (short field wins). Boost r
Why Debezium over dual-write?	Dual-write: app writes Postgres then ES; if ES write fails, inconsistency. Debezium reads WAL – cap
Completion vs edge_ngram?	Completion: FST in memory, O(1), lowest latency. Edge_ngram: index-time token expansion, larger
Zero-downtime reindex?	Create new index with new mapping; reindex (copy data); atomically swap alias. App uses alias – tra
What is a percolator?	Reverse search: store queries, then check incoming documents against all stored queries. Used for